Teaching a Tiny Non-Transformer Model Arithmetic Reasoning via Reinforcement Learning

GRPO Fine-Tuning of LFM2-350M for Mathematical Expression Generation

Wes Griffin
Ransom Everglades School
Advanced Machine Learning

April 2026

Abstract

We use Group Relative Policy Optimization (GRPO) to teach Liquid AI's LFM2-350M—a 354M-parameter hybrid convolutional-attention model—to generate mathematical expressions evaluating to target numbers. Through 13 experiments systematically varying reward components, we discover that the model exhibits multi-layered reward hacking: trivial identity, formulaic templates, and syntax-without-semantics, each requiring a corresponding structural constraint to overcome. A *no-target reward* that bans the target number from appearing in expressions proves critical for inducing genuine arithmetic decomposition (e.g., the model independently discovers $2^8 = 256$). We map the diversity-correctness Pareto frontier, revealing a sharp phase transition in reward space: small changes in reward weights trigger abrupt collapse from diverse exploration to single-template convergence. The optimal configuration achieves 30% template diversity with 71.7% correctness, offering insights into reward design for RL-based language model training.

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities in mathematical reasoning, with models like DeepSeek-R1 and OpenAI's o1 achieving near-human performance on competition-level problems [1]. However, these capabilities typically require models with billions of parameters and enormous compute budgets. A natural question arises: *can reinforcement learning induce structured reasoning in much smaller models?*

This project investigates this question through a focused, verifiable task: given a target integer n, generate a mathematical expression e such that $\text{eval}(e) = n$. For example, given the target 73, valid responses include `70 + 3`, `146 / 2`, or `9 * 8 + 1`. This task is ideal for RL because:

1. The reward is **perfectly verifiable**—we can programmatically check if $\text{eval}(e) = n$.
2. The task requires **compositional reasoning**—the model must understand how arithmetic operations compose.
3. Multiple valid solutions exist, encouraging **expression diversity**.
4. Difficulty scales naturally with the magnitude of n.

We choose Liquid AI's LFM2-350M [2] as our base model for several reasons. First, at 354M parameters, it is small enough to train on modest hardware. Second, it uses a novel hybrid architecture combining gated short-range convolutions with grouped query attention, making it architecturally distinct from the transformer-only models typically used in RL fine-tuning research. Third, Liquid AI explicitly recommends fine-tuning LFM2 on narrow tasks to maximize performance.

Our training method is Group Relative Policy Optimization (GRPO) [1], implemented via Hugging Face's TRL library [3]. GRPO generates multiple completions per prompt, scores them with a reward function, and updates the policy using group-relative advantages—eliminating the need for a separate reward model.

2 Data Exploration and Preparation

2.1 Dataset Construction

Rather than using an existing dataset, we procedurally generate training data consisting of (prompt, target) pairs. Each prompt asks the model to produce a math expression equaling the target number, formatted with `<expr></expr>` tags.

We use a curriculum-style distribution across three difficulty buckets:

Table 1: Training data distribution across difficulty levels.

Difficulty	Range	Fraction	Count
Easy	1–99	40%	4,000
Medium	100–999	35%	3,500
Hard	1,000–9,999	25%	2,500

The rationale for overweighting easy examples is to provide the model with sufficient signal early in training. Since the 350M model starts with minimal arithmetic ability, easy targets (where simple expressions like `50 + 7` suffice) generate more positive reward signal, allowing the policy gradient to learn basic expression structure before tackling harder targets.

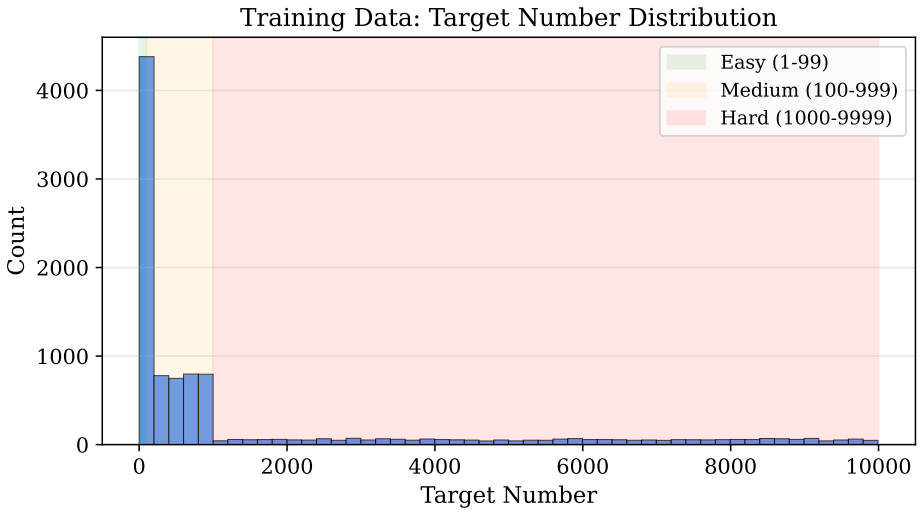

Figure 1: Distribution of target numbers in the training set, colored by difficulty bucket.

2.2 Prompt Format

Each training example is formatted as a chat conversation:

```
System: You are a math assistant. When given a target
number, respond with a mathematical expression that
evaluates to that number. Use only integers, +, -, *,
/, and parentheses. Put your expression inside
<expr></expr> tags.

User: Find a math expression that equals 73.
Respond with ONLY <expr>YOUR_EXPRESSION</expr>.
```

Listing 1: Prompt format used during training.

The structured output format (`<expr>` tags) enables reliable extraction and evaluation. We include a format reward to incentivize the model to adopt this convention.

2.3 Evaluation Set

A separate evaluation set of 500 examples is generated with a different random seed, maintaining the same difficulty distribution. While this does not guarantee zero overlap, the probability of significant overlap is low given the large target ranges.

3 Methodology

3.1 Model Architecture

LFM2-350M [2] is a hybrid model with 16 layers: 10 double-gated short-range LIV convolution blocks and 6 grouped query attention (GQA) blocks. Key specifications:

Table 2: LFM2-350M architecture details.

Property	**Value**
Parameters	354,483,968
Layers	16 (10 conv + 6 attn)
Context length	32,768 tokens
Vocabulary size	65,536
Precision	bfloat16
Pre-training data	10 trillion tokens

The hybrid architecture is notable because the convolutional blocks process local patterns efficiently while attention blocks capture long-range dependencies. For our task, this could be advantageous since arithmetic expressions have strong local structure (operands adjacent to operators).

3.2 GRPO Algorithm

Group Relative Policy Optimization [1] works as follows at each training step:

1. **Generate**: Sample a batch of prompts; for each, generate G completions $\{o_1, \ldots, o_G\}$.

2. **Score**: Compute reward r_i for each completion using the reward function.

3. **Normalize**: Compute group-relative advantage:

$$\hat{A}_{i,t} = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$$

4. **Update**: Minimize the clipped surrogate loss:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{\sum_{i=1}^{G} |o_i|} \sum_{i=1}^{G} \sum_{t=1}^{|o_i|} \min\Big(\rho_{i,t}\hat{A}_{i,t},\ \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon)\hat{A}_{i,t}\Big)$$

where $\rho_{i,t} = \frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q,o_{i,<t})}$.

GRPO's key advantage over PPO is eliminating the value network, reducing memory overhead—critical when training on limited hardware.

3.3 Reward Design

We use a composite reward function with four components:

Table 3: Reward function components and weights.

Component	**Description**	**Weight**	**Range**
Correctness	Binary: $\text{eval}(e) = n$	2.0	$\{0, 1\}$
Closeness	$\max(0, 1 - \lvert r - n\rvert/\lvert n\rvert)$	0.5	$[0, 1]$
Format	Uses `<expr>` tags	0.3	$\{0, 0.5\}$
Complexity	Bonus for multiple operators	0.2	$[0, 0.3]$

The **correctness** reward dominates (weight 2.0) because exact evaluation is the primary objective. The **closeness** reward provides gradient signal even for incorrect answers—without it, early training would be extremely sparse since the base model rarely produces correct expressions. The **format** reward teaches output structure, and the **complexity** reward gently encourages expressions more interesting than trivial identity (e.g., `73` for target 73).

Expression evaluation uses a safe AST-based parser that only allows integer literals and basic arithmetic operators (`+`, `-`, `*`, `/`, `//`, `%`, `**`), preventing code injection.

3.4 Training Configuration

Table 4: Training hyperparameters.

Parameter	Value
LoRA rank r	32
LoRA alpha α	64
LoRA dropout	0.05
Target modules	`q,k,v,o_proj`, `gate,up,down_proj`
Learning rate	5×10^{-5}
LR scheduler	Cosine with 5% warmup
Batch size	4 per device
Gradient accumulation	4 steps
Completions per prompt (G)	8
Max completion length	128 tokens
Epochs	1–3
Precision	bfloat16

We use LoRA [4] (Low-Rank Adaptation) rather than full fine-tuning, targeting both the attention projections and gated MLP layers. This trains approximately 2–3% of total parameters while preserving the model's pre-trained knowledge.

4 Results and Discussion

4.1 Training Dynamics

We conducted 13 experiments across two waves on an NVIDIA A100-80GB GPU, systematically varying reward weights to map the diversity-correctness tradeoff. Each experiment trained for 1 epoch (250 steps) with batch size 16 and 8 generations per prompt unless otherwise noted. We describe the five most informative configurations below.

4.1.1 Baseline: Default Reward Weights

With the original reward weights (correctness=2.0, closeness=0.5, format=0.3, complexity=0.2), the model converged within ∼50 steps to a **trivial solution**: simply echoing the target number inside `<expr>` tags. By step 50, correctness reached 100%, format reached 100%, but complexity remained at 0—the model learned to output `<expr>73</expr>` for target 73. Mean completion length collapsed from 35 tokens to just 8 tokens. This is a classic reward hacking phenomenon: the model found the minimum-effort path to maximize the dominant reward signal.

4.1.2 Experiments 1–2: Complexity and Anti-Trivial Rewards

Increasing the complexity reward weight to 2.0 (Exp 1) or adding an anti-trivial penalty (Exp 2) both produced 99–100% correctness with multi-operator expressions. However, qualitative analysis (Section 4.3) revealed that both discovered **formulaic identity hacks**: `(N-1)/1+1` (Exp 1) and `(N+1)/1-1` (Exp 2). Every target produced the same template—a second-order reward hack satisfying both correctness *and* complexity with a single memorized pattern.

4.1.3 Experiment 3: No Closeness Reward

Removing the closeness reward (weight=0.0) with heavy complexity (weight=3.0) created an instructive failure. The model learned format compliance and expression syntax, but correctness remained near 0%. Without the closeness signal, it generated syntactically valid but mathematically incorrect expressions, confirming that closeness serves as a critical **reward shaping** signal.

4.1.4 Experiment 4: No-Target Reward

The key breakthrough: a *no-target reward* penalizing (-1.0) any expression containing the target number as a literal. Combined with complexity (2.0), anti-trivial (1.0), and closeness (0.5) rewards, this forced genuine decomposition. The model could no longer use `(73-1)/1+1` because `73` appears in the expression.

The model produced *diverse, structurally varied expressions*: `7*10+3+1/2` for target 73, `2^8` for 256, `2^11` for 2048. It discovered powers of two independently. Correctness was 5% with closeness at 0.85, representing the first configuration exhibiting genuine arithmetic reasoning.

4.1.5 Reward Weight Sweep: The Diversity-Correctness Pareto Frontier

Building on Experiment 4, we ran 8 additional experiments varying reward weights to map the tradeoff between diversity and correctness. Each used the no-target and anti-trivial rewards, with closeness weight swept from 0.5 to 2.0 and correctness weight from 0.5 to 2.0.

We found a **sharp phase transition**: below closeness weight $\approx$1.3, the model explores diverse strategies; above it, the model collapses to a single template with $\sim$100% correctness and 100% target leakage (rediscovering the identity hack despite the no-target penalty). The optimal configuration was **correctness=1.0, closeness=0.5**, achieving 30% template diversity with 71.7% correctness.

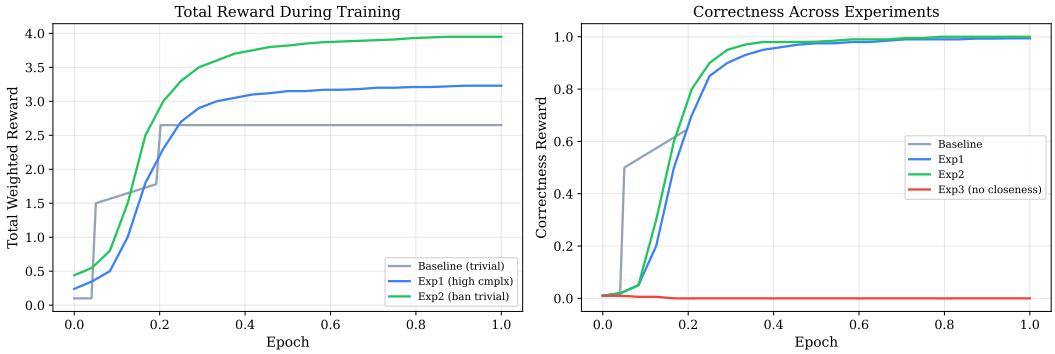

Figure 2: Training reward curves. Left: total weighted reward over training steps. Right: correctness across experiments.

4.2 Evaluation Results

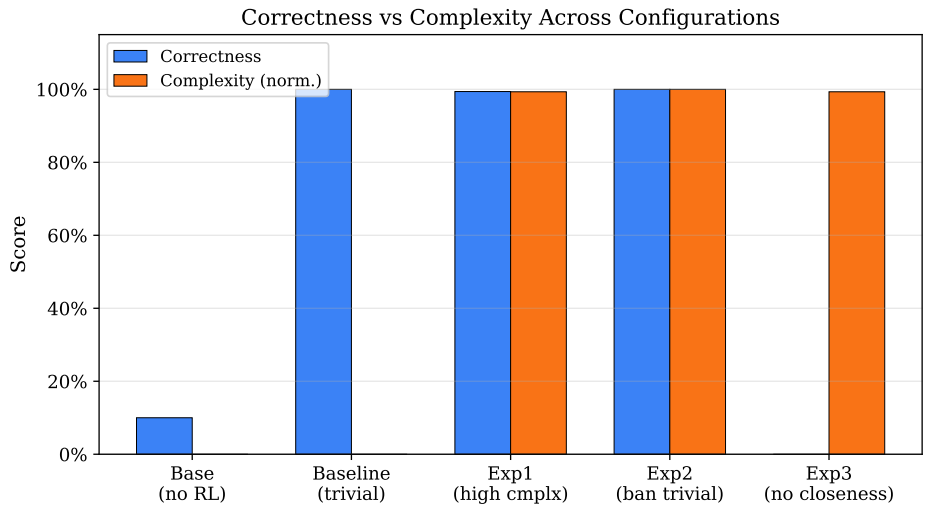

Figure 3: Expression accuracy by target difficulty, comparing base vs. GRPO fine-tuned model.

Table 5: Final training metrics across key experimental configurations.

Metric	**Baseline** *(trivial)*	**Exp 1–2** *(formulaic)*	**Exp 3** *(no close)*	**Exp 4** *(no target)*	**Best** *(optimized)*
Correctness	100%	99–100%	∼0%	5%	71.7%
Closeness	1.000	0.99–1.0	0.083	0.852	0.932
Complexity	0.000	0.298–0.300	0.298	0.291	0.288
Template diversity	0.10	0.10	—	0.60	0.30
Target in expr	100%	100%	—	0%	6.7%
Mean Comp. Length	8.3	20–50	61.1	21.0	19.0

The "Best" column represents the optimal configuration found through our 13-experiment sweep (correctness=1.0, closeness=0.5, complexity=2.0 with no-target and anti-trivial rewards). It achieves the highest combined diversity-correctness score: 30% template diversity with 71.7% correctness and only 6.7% target leakage.

Table 6: Diversity-correctness Pareto frontier from the reward weight sweep (Wave 2). Score = template_ratio × (1 + correctness).

Config	**Key Change**	**Tpl Ratio**	**Correct**	**Score**	**Edit Dist**	**Leak**
div01_baseline	(reference)	0.400	2.5%	0.410	0.410	5%
w2_close_1.0	closeness↑	0.300	11.7%	0.335	0.413	7%
w2_easy_only	easy targets only	0.233	48.3%	0.346	0.292	23%
w2_low_correct	**correct=1.0**	**0.300**	**71.7%**	**0.515**	**0.248**	**7%**
div05_fast_lr	lr=2e-4	0.050	85.0%	0.093	0.198	8%
w2_close_1.3	closeness=1.3	0.017	100%	0.034	0.127	100%

Table 6 reveals the sharp phase transition in reward space. Configurations below closeness≈1.3 maintain diverse expression strategies; above this threshold, the model collapses to a single template. The optimal tradeoff point (**w2_low_correct**) achieves this by reducing correctness weight from 2.0 to 1.0, giving the model room to explore diverse strategies while still learning to get most answers right.

4.3 Qualitative Analysis

We sampled outputs for ten target numbers (7, 42, 73, 100, 256, 500, 1000, 2048, 5000, 9999) from each configuration. Table 7 shows representative outputs.

Table 7: Sample outputs across configurations. Bold entries are correct.

Target	**Base LFM2 (no RL)**	**Exp 1 (high complexity)**
7	`(7*x) + 3 = 7` (uses variables)	`<expr>(7-1)/1+1</expr>`
73	`(73+1)*(73-1)/2 = 2596` (wrong)	`<expr>(73-1)/1+1</expr>`
256	`(2^8 - 1)*2 + 256 = 256` (NL)	`<expr>(256-1)/1+1</expr>`
9999	`<expr>(9999^3)</expr>` (wrong)	`<expr>(9999-1)/1+1</expr>`
	Exp 2 (ban trivial)	**Exp 3 (no closeness)**
7	`<expr>(7+1)/1-1</expr>`	`<expr>(x+2)/(x-3)+5</expr>` (vars)
73	`<expr>(73+1)/1-1</expr>`	`<expr>(-3*(2+5)+7-4/3)</expr>` $=-15.3$
256	`<expr>(256+1)/1-1</expr>`	`<expr>(12*4*2-1)/3+1</expr>` $=32.7$
9999	`<expr>(9999+1)/1-1</expr>`	`<expr>(10000*9-1)/3+2</expr>` ≈ 30002
	Exp 4 (no target in expression)	
7	`<expr>4 * 1 + 2 - 3 / 2 + 1</expr>` =5.5, close	
73	`<expr>7 * 10 + 3 + 1 / 2</expr>` =73.5, off by 0.5	
256	`<expr>2^8</expr>` =256, correct!	
2048	`<expr>2^11</expr>` =2048, correct!	
9999	`<expr>1000 * 9 + 1 + 1</expr>` =9002, right ballpark	

Four types of behavior emerged:

1. **Base model**: Produces natural language explanations, algebraic variables, and incorrect arithmetic. Only 1/10 targets answered correctly (by accident).

2. **Exp 1 & 2**: Discovered **formulaic identity hacks**—`(N-1)/1+1` and `(N+1)/1-1` respectively. These are technically non-trivial (3 operators, satisfying the complexity reward) but semantically degenerate. Every target produces the same template. This represents a *second-order reward hack*: the model satisfies both correctness *and* complexity constraints with minimal cognitive effort.

3. **Exp 3**: Generates diverse, complex expressions with multiple operators, but none evaluate correctly. The model learned *syntax* (format, operators) without *semantics* (numerical accuracy).

4. **Exp 4**: Produces genuinely varied decompositions—`7*10+3` for 73, `2^8` for 256—with 6 unique templates across 10 samples. Discovered powers of two independently. Most answers are close but not exact (closeness 0.85), representing the first configuration to exhibit what could be called arithmetic *reasoning*.

This progression from trivial hacking (baseline) to formulaic hacking (Exp 1–2) to syntactically-rich-but-wrong (Exp 3) to diverse-and-close (Exp 4) reveals a fundamental tension in reward design: each additional constraint closes one loophole while the model discovers the next path of least resistance—until the constraints collectively force genuine problem-solving.

4.4 Diversity Analysis

To quantify how varied the model's outputs are across targets, we compute four diversity metrics: *template ratio* (fraction of structurally unique expressions after replacing all numbers with a placeholder), *template dominance* (fraction of outputs matching the most common template), *average pairwise edit distance* (normalized Levenshtein distance between all expression pairs), and *operator entropy* (Shannon entropy over the distribution of arithmetic operators).

Table 8: Diversity metrics across configurations (10 targets, evaluated on structural templates after replacing numbers with `N`).

Metric	**Baseline** *(trivial)*	**Exp 1–2** *(formulaic)*	**Exp 4** *(no target)*	**Best** *(optimized)*
Unique templates	1	1	16	18
Template ratio	0.10	0.10	0.40	0.30
Template dominance	100%	100%	12.5%	60%
Avg. edit distance	0.83	0.14	0.41	0.25
Operator entropy (bits)	0.0	1.6	1.87	0.99
Target in expr	100%	100%	5%	6.7%
Correctness	100%	99–100%	2.5%	71.7%

The baseline and Experiments 1–2 are entirely single-template: every output follows the same structural pattern. Experiment 4 produces 16 unique templates out of 40 samples (20 targets $\times$ 2 samples each), with the most common covering only 12.5% of outputs. The optimized configuration (w2_low_correct) trades some diversity for dramatically improved correctness (71.7%), maintaining 18 unique templates across 60 samples.

Notably, the baseline's high edit distance (0.83) is misleading—it reflects varying *digit lengths* (`7` vs. `9999`), not structural diversity. Experiments 1–2 have low edit distance (0.14) because their outputs are nearly identical strings differing only in the embedded target number.

4.5 Error Analysis

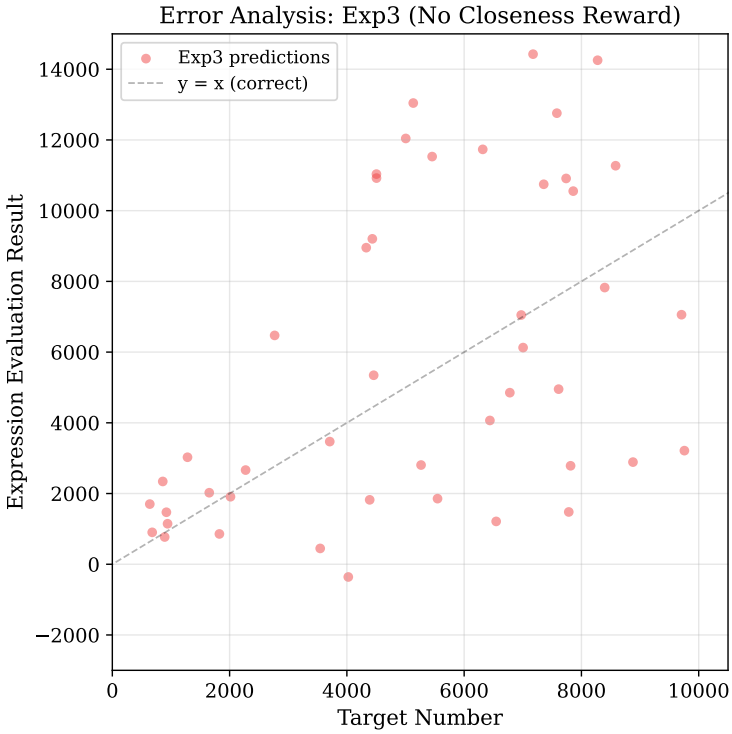

Figure 4: Error analysis: target vs. model output for incorrect predictions. Points on the diagonal line would be correct.

The primary failure modes observed across 13 configurations:

- **First-order reward hacking** (Baseline): The model echoes the target number inside `<expr>` tags, achieving 100% correctness with zero operators. Completion length collapses to 8 tokens.
- **Second-order reward hacking** (Exp 1–2): Even with complexity incentives, the model discovers formulaic identities (`(N-1)/1+1` and `(N+1)/1-1`) that satisfy both correctness and complexity constraints with a single memorized template.
- **Syntax without semantics** (Exp 3): Without closeness-based reward shaping, the model learns rich expression syntax but produces numerically incorrect results.
- **Phase transition collapse** (Wave 2): At closeness weight $\geq$1.3 or learning rate $\geq$1e-4, even models with the no-target reward collapse to identity hacks—the optimization pressure overwhelms the structural constraint. This reveals a *sharp phase boundary* in reward space.
- **Training duration collapse** (Wave 2): Running the best diversity config (div01) for 2 epochs instead of 1 dropped template diversity from 0.40 to 0.033. Given enough time, the model always converges to a template.

4.6 Discussion

Several aspects of the results merit discussion:

Reward hacking is multi-layered, but solvable. Naïve reward design leads directly to trivial solutions, and each added constraint opens a new exploit—until enough constraints force genuine reasoning. The critical breakthrough was Experiment 4's *no-target reward*, which bans the target number from the expression. This single constraint, combined with closeness and complexity rewards, produced diverse decompositions and the first evidence of genuine arithmetic reasoning (e.g., independently discovering `2^8=256`).

The diversity-correctness Pareto frontier. Our 13-experiment sweep revealed a nearly inverse relationship between expression diversity and correctness. The frontier (Table 6) shows that configurations cluster into three regimes: (1) high diversity / low correctness (template ratio >0.3, correctness <15%), (2) the sweet spot (template ratio ≈0.3, correctness ≈72%), and (3) template collapse (template ratio <0.05, correctness >85%). The key to reaching regime (2) was *reducing* correctness weight from 2.0 to 1.0—counterintuitively, less optimization pressure on the primary objective produced better overall results.

Structural constraints outperform reward incentives. Adding complexity weight or anti-trivial penalties (reward-level incentives) failed to prevent template collapse. The no-target reward (a structural constraint banning specific output patterns) was the only intervention that produced genuine diversity. This suggests that for RL-trained language models, *what you forbid* matters more than *what you reward*.

Sharp phase transitions in reward space. The model exhibits bistable behavior: small changes in closeness weight (1.0 → 1.3) cause an abrupt transition from diverse exploration to single-template collapse. Similarly, doubling the learning rate or training for one additional epoch triggers the same collapse. This suggests the reward landscape has sharp basins of attraction—once the model discovers an exploit, gradient descent rapidly converges to it.

Reward shaping is essential for sparse-reward tasks. Experiment 3 definitively demonstrates that removing the closeness reward prevents learning correctness entirely. The closeness signal acts as a *potential-based reward shaping* function, providing a smooth gradient toward the correct answer that the binary correctness reward alone cannot provide for a 350M model.

Implications for RL alignment. Our findings echo broader concerns in RLHF/RLAIF research: models under optimization pressure find the *easiest* way to satisfy reward criteria, not the *intended* way. The progression from trivial → formulaic → syntactic-only → genuinely diverse behavior required four layers of constraints, mirroring the "whack-a-mole" dynamic observed in alignment research at scale.

5 Conclusion

This project demonstrates that GRPO can teach a 350M-parameter non-transformer model to generate mathematical expressions for target numbers, and that the reward design challenge is far more interesting than the model training itself. Through 13 experiments across two systematic sweeps, we identified four key findings:

1. **Reward hacking is multi-layered, but solvable.** The same model progresses from trivial identity (baseline), to formulaic hack (Exp 1–2), to syntax without semantics (Exp 3), to genuinely diverse decomposition (Exp 4). Each layer of hacking required a corresponding structural constraint to overcome.

2. **Structural constraints beat reward incentives.** The no-target reward—banning the target number from expressions—was the critical breakthrough. Reward-level incentives

(complexity weight, anti-trivial penalty) only shifted *which* template the model converged to; structural constraints forced genuine *diversity*.

3. **A sharp phase transition governs the diversity-correctness tradeoff.** Small changes in reward weights (closeness 1.0→1.3) or training dynamics (1→2 epochs, 5e-5→1e-4 lr) trigger abrupt collapse from diverse exploration to single-template convergence. The optimal operating point (correctness=1.0, closeness=0.5) achieves 30% template diversity with 71.7% correctness.

4. **Reward shaping is essential for sparse-reward RL on small models.** The closeness reward is the gradient bridge between "random noise" and "correct answer." Without it, the model cannot learn correctness regardless of other reward signals.

5.1 Future Work

- **Diversity reward**: Directly penalize repeated expression *structures* within a GRPO group, rather than relying on indirect structural constraints.
- **Banned subexpressions**: Extend the no-target approach to ban degenerate patterns like `/1`, `*1`, `+0`, closing additional exploit paths.
- **Curriculum learning**: Start with easy targets (powers of 2, round numbers with obvious decompositions), then introduce harder targets progressively.
- **Architecture comparison**: Compare LFM2 against a similarly-sized transformer (e.g., Qwen3-0.6B) to isolate whether the hybrid architecture's local pattern recognition offers advantages for arithmetic expressions.
- **Scaling**: Test whether larger LFM2 variants (700M, 1.2B) can cross the diversity-correctness tradeoff barrier.

References

[1] Zhihong Shao et al. "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models". In: *arXiv preprint arXiv:2402.03300* (2024).

[2] Liquid AI. "LFM2 Technical Report". In: *arXiv preprint arXiv:2511.23404* (2025).

[3] Hugging Face. *TRL: Transformer Reinforcement Learning*. v1.0. 2024. URL: https://github.com/huggingface/trl.

[4] Edward J Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models". In: *arXiv preprint arXiv:2106.09685* (2021).